ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ВНУТРІШНІХ СПРАВ

Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №4

з дисципліни «Об’єктно-орієнтовне програмування»

Тема: «Абстрактні класи і методи (модуль abc)»

Виконав:

курсант 2-го курсу

ННІ 4-25-203 інф

Кожевніков К.О.

Прийняв:

Доцент кафедри ІСТ, к.т.н.,
Слободянюк О.В.

КАМ’ЯНЕЦЬ-ПОДІЛЬСЬКИЙ – 2026

ВАРІАНТ 6. Медичний центр

Завдання:

Класи для роботи: Service, Appointment, Client. Створіть абстрактний Appointment із методом preparation_note(); конкретні класи мають повертати різні примітки. Використовуйте вигадані дані.

КОД РОЗВ'ЯЗКУ

```
from abc import ABC, abstractmethod

class Service:
    """Модель медичної послуги."""

    def __init__(self, name, duration_min):
        self.name = name
        self.duration_min = duration_min

    def show_service(self):
        """Повертає інформацію про послугу."""
        return f"{self.name} ({self.duration_min} хв)"

class Client:
    """Модель пацієнта (клієнта) медичного центру."""

    def __init__(self, full_name, phone):
        self.full_name = full_name
        self.phone = phone

    def show_client(self):
        """Повертає інформацію про клієнта."""
        return f"{self.full_name} (тел: {self.phone})"

class Appointment(ABC):
    """Абстрактний базовий клас запису на прийом із контрактом
для нащадків."""

    valid_statuses = {"підтверджено", "завершено", "скасовано"}
    created_count = 0  # Атрибут класу для підрахунку створених
записів

    def __init__(self, number, time, service, client,
status="підтверджено"):
        self.number = number
        self.time = time
        self.service = service  # Екземпляр класу Service
        self.client = client    # Екземпляр класу Client
```

```
            self._status = "підтверджено"  # Початковий стан перед
викликом setter
            self.status = status           # Запис через setter із
перевіркою

            Appointment.created_count += 1

        @property
        def status(self):
            """Повертає поточний статус із захищеного внутрішнього
поля."""
            return self._status

        @status.setter
        def status(self, value):
            """Дозволяє встановити лише допустимий статус."""
            if value not in self.valid_statuses:
                raise ValueError(f"Невідомий статус: {value}")
            self._status = value

        @abstractmethod
        def preparation_note(self):
            """Абстрактний метод: кожен похідний клас зобов'язаний
надати власну реалізацію."""
            pass

        def summary(self):
            """Конкретний спільний метод базового класу для
виведення основної інформації."""
            return (
                f"Запис №{self.number} | Час: {self.time} | "
                f"Клієнт: {self.client.show_client()} | "
                f"Послуга: {self.service.show_service()} | Статус:
{self.status}"
            )

    class Consultation(Appointment):
        """Похідний клас для консультаційних прийомів лікаря."""

        def __init__(self, number, time, service, client,
doctor_office, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.doctor_office = doctor_office  # Номер кабінету
лікаря

        def preparation_note(self):
            """Обов'язкова реалізація абстрактного методу для
консультації."""
```

```
            return f"Консультація у каб. №{self.doctor_office}: 
очікуйте під кабінетом, прийдіть за 10 хв."

    class DiagnosticVisit(Appointment):
        """Похідний клас для діагностичних процедур (УЗД тощо)."""

        def __init__(self, number, time, service, client, 
equipment_code, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.equipment_code = equipment_code  # Інвентарний код 
обладнання

        def preparation_note(self):
            """Обов'язкова реалізація абстрактного методу для 
діагностики."""
            return f"Діагностика (обладнання код: 
{self.equipment_code}): дотримуйтеся спеціальних інструкцій лікаря 
перед процедурою."

    # Демонстрація роботи абстрактних класів та поліморфізму
    if __name__ == "__main__":
        # Створюємо об'єкти послуг
        s1 = Service("Консультація терапевта", 30)
        s2 = Service("УЗД черевної порожнини", 40)

        # Створюємо вигадані об'єкти клієнтів
        c1 = Client("Олександр Т.", "+380501112233")
        c2 = Client("Марія Д.", "+380672223344")

        # 1. Перевірка неможливості створення екземпляра 
абстрактного класу Appointment:
        try:
            # Спроба створення об'єкта абстрактного класу викличе 
TypeError
            invalid_app = Appointment(999, "2026-11-03 12:00", s1, 
c1)
        except TypeError as e:
            print(f"Помилка створення абстрактного класу 
(очікувано): {e}")

        # 2. Створення коректних об'єктів похідних класів
        appointments = [
            Consultation(1, "2026-11-03 09:00", s1, c1, "204", 
"підтверджено"),
            DiagnosticVisit(2, "2026-11-03 10:30", s2, c2, "EQ-
9988", "підтверджено")
        ]
```

```
        print("\n=== Демонстрація роботи абстрактного базового
класу та його нащадків ===")
        for app in appointments:
            # Виклик спільного конкретного методу та поліморфного
абстрактного методу
            print(app.summary())
            print(f"   -> Інструкція: {app.preparation_note()}")
            print("-" * 70)

        print(f"Загалом успішно створено записів:
{Appointment.created_count}")
```

РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

